Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

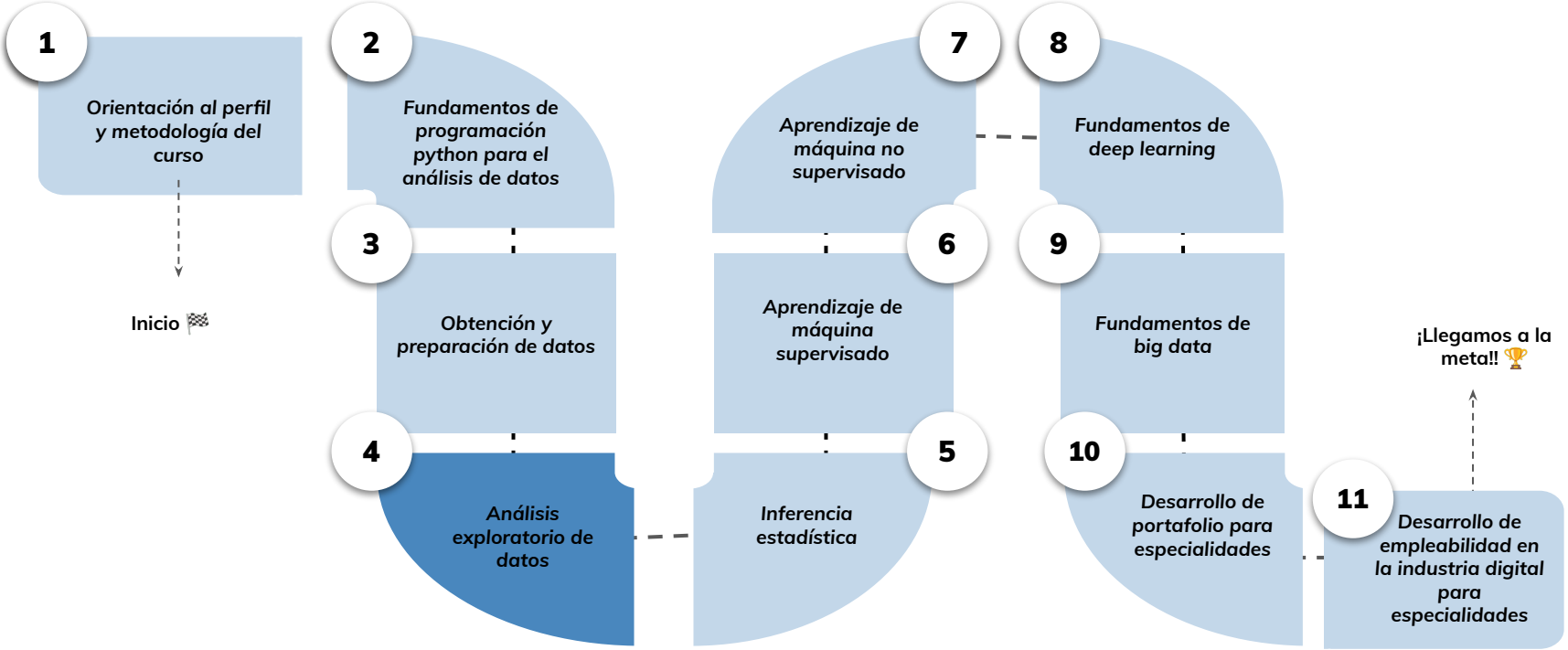
› Análisis visual de datos- Parte I

Aprendizaje Esperado 5: Aplicar técnicas de análisis visual de los datos utilizando la librería seaborn para la representación gráfica de la información.

alkemy

Hoja de ruta

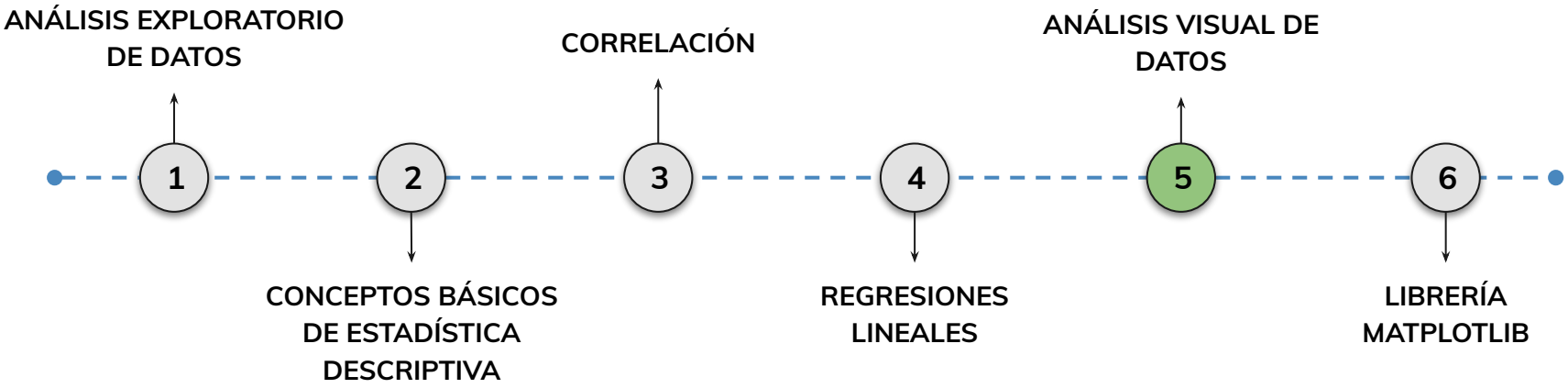
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Visualización y análisis gráfico de relaciones entre variables con Python.

Exploraremos el uso de la librería Seaborn para generar gráficos estadísticos que nos permitan identificar patrones, relaciones y comportamientos entre variables. Aplicaremos visualizaciones para validar hipótesis, detectar correlaciones y ajustar modelos lineales simples de forma visual.

- **Fundamentos de visualización estadística con Seaborn**
  - **Características de Seaborn y comparación con Matplotlib**
  - **Tipos de gráficos: histogramas, scatterplots, boxplots y heatmaps**
- **Gráficos orientados a análisis de regresión**
  - **Uso de regplot, jointplot y pairplot**
  - **Personalización, comparación y análisis de regresión visual**

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender la utilidad de la visualización en el análisis exploratorio de datos.
- Conocer los tipos de gráficos que ofrece la librería Seaborn y sus principales usos.
- Representar visualmente la relación entre variables mediante gráficos de dispersión y regresión.
- Interpretar patrones visuales que sugieran correlación lineal o no lineal.
- Utilizar herramientas visuales como apoyo para decisiones basadas en datos.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Aplicamos modelos de regresión lineal simple y múltiple con Python.*

→ *Calculamos métricas de evaluación como R^2, MAE y MSE.*

→ *Interpretamos los coeficientes del modelo y verificar los supuestos de regresión.*

→ *Comparamos modelos utilizando R^2 ajustado, AIC y BIC.*

Análisis visual de datos

Visualización de Datos con Seaborn en Python

En la era del big data, el análisis visual de los datos se ha convertido en una herramienta esencial para la comprensión, exploración y comunicación de información. Una correcta representación gráfica no solo permite identificar patrones, relaciones o anomalías en los datos, sino que también facilita la toma de decisiones fundamentadas. En el ecosistema de Python, la librería Seaborn se destaca por su capacidad para generar gráficos estadísticos de alta calidad con un enfoque intuitivo y atractivo visualmente.

alkemy

Características de la librería Seaborn

Seaborn es una librería de visualización estadística construida sobre Matplotlib, que proporciona una interfaz de alto nivel para crear gráficos informativos y atractivos. Sus principales características son:

Integración con pandas

Trabaja perfectamente con DataFrames y Series.

Estética mejorada

Gráficos más profesionales y pulidos por defecto.

Gráficos estadísticos

Incorpora funcionalidades estadísticas como líneas de regresión, intervalos de confianza, distribución de frecuencias, etc.

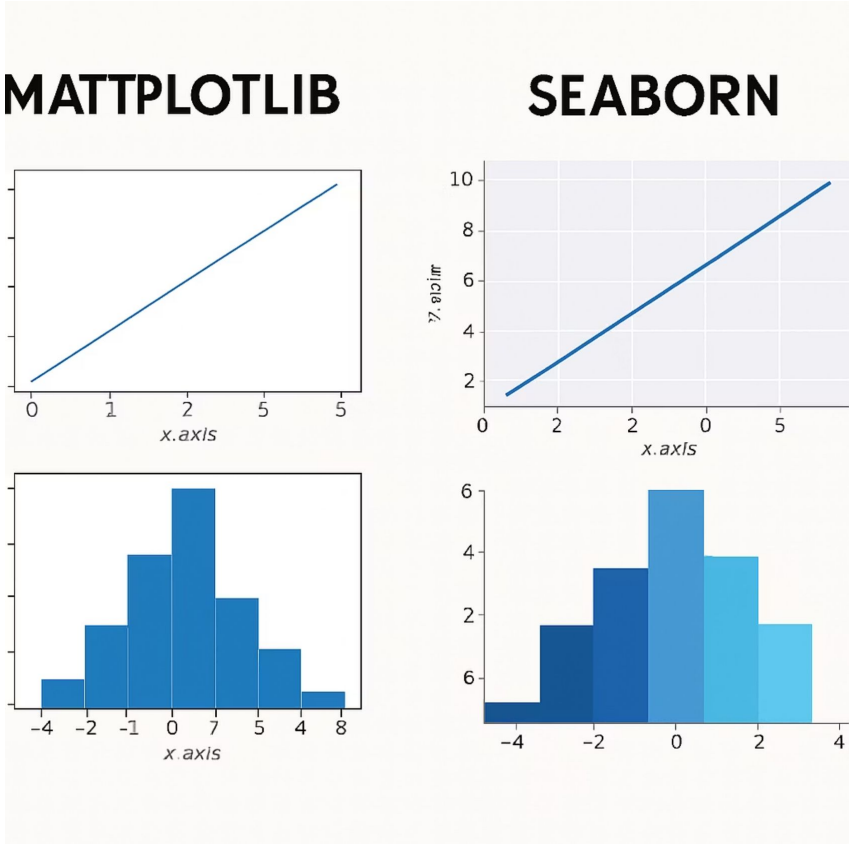
Paletas de colores integradas

Permite aplicar fácilmente paletas cromáticas temáticas.

Soporte para múltiples ejes

Con PairGrid y FacetGrid se pueden crear visualizaciones complejas en grillas.

Ventajas frente a Matplotlib



A diferencia de Matplotlib, que requiere mayor configuración manual para personalizar los gráficos, Seaborn ofrece funciones preconfiguradas que permiten obtener resultados impactantes con pocas líneas de código.

- Menos código para gráficos profesionales
- Integración nativa con pandas
- Estilos predefinidos atractivos
- Funciones estadísticas incorporadas

alkemy

Importación de la librería

Para comenzar a utilizar Seaborn, es necesario importarla junto con sus dependencias más comunes:

En la mayoría de los casos, Seaborn ya está disponible si se trabaja en entornos como Jupyter Notebook o Google Colab. De no estar instalada, se puede instalar mediante:

```
pip install seaborn
```

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

Instalación de Seaborn

También es habitual configurar el estilo general de los gráficos desde el inicio:

```
sns.set(style="darkgrid")
```

Esto establece un fondo gris con líneas de grilla, muy útil para visualizar tendencias.

Tipos de gráfico en Seaborn

Seaborn permite generar una amplia variedad de gráficos, agrupados según la naturaleza del análisis:

Tipo de gráfico	Funciones asociadas	Uso principal
Distribución univariada	distplot, histplot, kdeplot	Distribución de una variable numérica
Relación bivariada	scatterplot, regplot, jointplot	Relación entre dos variables
Comparación categórica	barplot, countplot, boxplot, violinplot	Comparación entre grupos categóricos
Correlación/matriz	heatmap, clustermap	Visualización de matrices y relaciones
Múltiples gráficos	pairplot, PairGrid, FacetGrid	Visualización de múltiples variables

Parámetros comunes

Cada función tiene sus parámetros propios, pero también comparte configuraciones generales como hue, palette, style, size, lo cual permite personalizar completamente las visualizaciones.

hue

Variable categórica para diferenciar por colores

palette

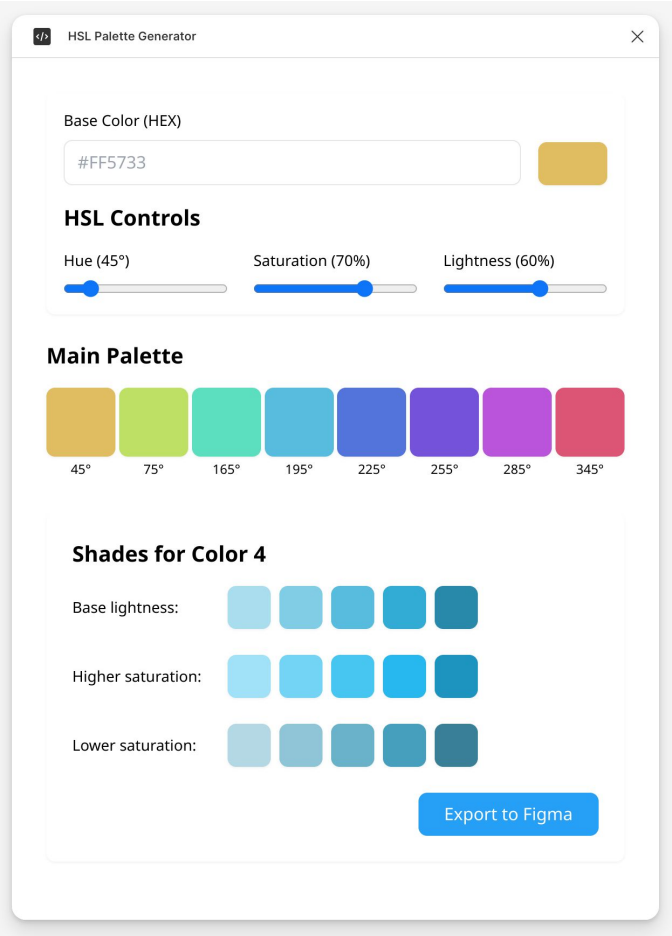
Esquema de colores a utilizar

style

Estilo visual del gráfico

size

Tamaño de los elementos gráficos



alkemy

Gráficos de distribución: distplot

Nota importante: distplot está obsoleta desde Seaborn v0.11. Se recomienda utilizar displot() o histplot().

distplot era la función tradicional de Seaborn para graficar histogramas de frecuencia con una curva de densidad ajustada. Se usaba así:

```
sns.distplot(df['edad'])
plt.show()
```

Hoy en día, se recomienda:

```
sns.histplot(df['edad'], kde=True)
plt.show()
```

Parámetros importantes:

- kde=True: activa la curva de densidad.
- bins=30: cantidad de intervalos.
- color='skyblue': color del histograma.

alkemy

Comparación de funciones de distribución

Función	Vigente	Curva KDE	Personalización fácil
distplot	No	Sí	Media
histplot	Sí	Sí	Alta
displot	Sí	Sí	Muy alta (grid opcional)

Conclusión: para análisis actuales, es recomendable usar histplot para gráficos univariantes o displot para múltiples subgráficos.

Gráficos de dispersión y correlación: jointplot

jointplot() combina un gráfico de dispersión con histogramas marginales, mostrando cómo se relacionan dos variables numéricas y su distribución individual.

```
sns.jointplot(data=df, x='altura', y='peso', kind='scatter')
```

alkemy

Tipos de jointplot

scatter

Dispersión tradicional

kde

Densidad bivariada

hex

Hexbin (agrupamiento por densidad)

reg

Dispersión con línea de regresión

Ventajas:

- Muestra relaciones y distribuciones al mismo tiempo.
- Fácil de interpretar para detectar patrones.

Ejemplo de jointplot con regresión

Ideal para estudios económicos, salud o cualquier análisis de correlación.

```
sns.jointplot(data=df, x='ingresos', y='gastos', kind='reg')
```

alkemy

Gráficos de correlación múltiple: pairplot

pairplot() es una de las funciones más poderosas de Seaborn para analizar la relación entre varias variables numéricas al mismo tiempo. Genera una matriz de gráficos de dispersión (scatterplots) para cada par de variables, y distribuciones en la diagonal.

```
sns.pairplot(df)
plt.show()
```

Parámetros importantes de pairplot

Parámetro	Función
hue	Distingue categorías con color
palette	Esquema de colores
kind	Tipo de gráfico (scatter, kde, etc.)
diag_kind	Tipo de gráfico en la diagonal (hist o kde)

alkemy

Ejemplo de pairplot con categoría

Este tipo de gráfico es ideal para análisis exploratorio de datos (EDA), especialmente en modelos de machine learning, donde se busca visualizar la relación entre múltiples variables a la vez.

```
sns.pairplot(df, hue='genero', palette='Set2')
```

alkemy

Gráficos de regresiones: regplot

regplot() permite representar gráficamente la relación entre dos variables continuas ajustando una línea de regresión lineal, además de visualizar los puntos de datos.

Características:

- Usa una regresión lineal por defecto.
- Admite ajuste de modelo con order (polinomial).
- Muestra el intervalo de confianza (por defecto al 95%).

```
sns.regplot(x='edad', y='ingresos', data=df)
```

alkemy

Personalización de regplot

Parámetro	Descripción
ci	Nivel de confianza (0 a 100)
line_kws	Personaliza la línea (color, grosor)
scatter_kws	Personaliza los puntos

```
sns.regplot(x='horas_estudio', y='nota_final', data=df, line_kws={'color':'red'})
```

Este gráfico es ideal cuando queremos ver si existe una tendencia creciente o decreciente entre dos variables, o si existe dispersión que justifique la no linealidad.

alkemy

Gráficos de variables categóricas

Estas funciones están diseñadas para comparar categorías. A continuación las analizamos en profundidad.

barplot

Representa el valor medio de una variable numérica para cada categoría.

countplot

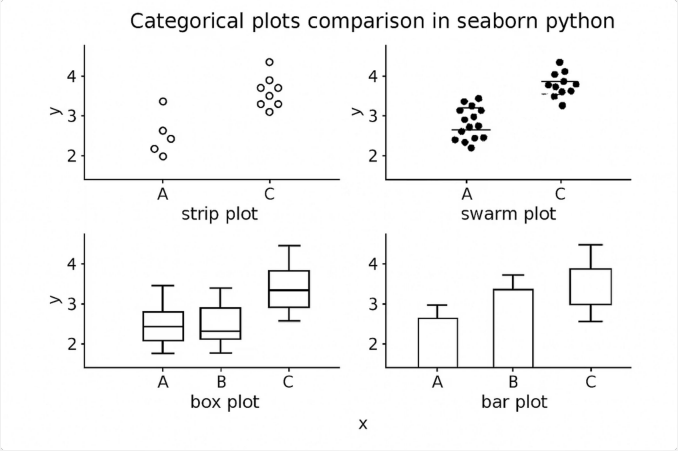
Cuenta la cantidad de observaciones por categoría.

boxplot

Muestra la mediana, cuartiles, valores atípicos y rango intercuartílico.

violinplot

Similar a boxplot, pero agrega la estimación de densidad (curva KDE).



alkemy

barplot: Gráfico de barras

Representa el valor medio de una variable numérica para cada categoría.

- Calcula automáticamente la media y el intervalo de confianza.
- Útil para comparar promedios entre grupos.

```
sns.barplot(x='carrera', y='promedio', data=df)
```

countplot: Gráfico de conteo

Cuenta la cantidad de observaciones por categoría.

- No requiere variable numérica.
- Ideal para ver la distribución de frecuencias.

```
sns.countplot(x='genero', data=df)
```

alkemy

boxplot: Diagrama de caja

Muestra la mediana, cuartiles, valores atípicos y rango intercuartílico.

```
sns.boxplot(x='region', y='ingresos', data=df)
```

Elemento	Significado
Caja	Rango intercuartílico (Q1 a Q3)
Línea	Mediana
Bigotes	Rango sin outliers
Puntos	Outliers

Permite detectar valores extremos fácilmente.

alkemy

violinplot: Diagrama de violín

Similar a boxplot, pero agrega la estimación de densidad (curva KDE), combinando información cuantitativa y distribución.

Diferencia principal:

- boxplot: muestra resumen estadístico
- violinplot: añade distribución completa

```
sns.violinplot(x='carrera', y='nota_final', data=df)
```

Comparativa de gráficos categóricos

Función	Uso principal	Datos requeridos
barplot	Comparar promedios	Numérico + categórico
countplot	Contar observaciones	Solo categórico
boxplot	Visualizar dispersión y outliers	Numérico + categórico
violinplot	Ver distribución + resumen estadístico	Numérico + categórico

alkemy

Gráficos de matrices: heatmap

heatmap() representa valores en forma matricial, usando color como variable cuantitativa. Es especialmente útil para visualizar matrices de correlación.

```
correlation = df.corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm')
```

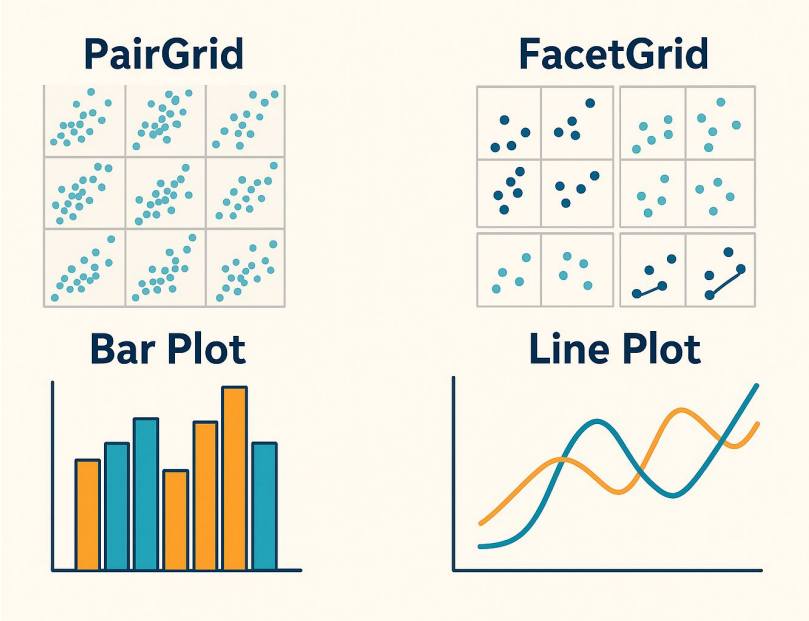
Parámetros clave de heatmap

Parámetro	Función
annot	Mostrar los valores sobre cada celda
cmap	Paleta de colores
linewidths	Líneas entre celdas
vmin, vmax	Escala de colores personalizada

Se utiliza ampliamente en análisis de variables para identificar relaciones significativas o multicolinealidad en modelos predictivos.

Grillas de gráficos: PairGrid y FacetGrid

Estas funciones permiten construir layouts complejos con múltiples gráficos de forma más flexible que pairplot() o displot().



PairGrid

Permite definir con mayor control qué función aplicar en cada parte de la matriz de gráficos.

FacetGrid

Divide el dataset en subgráficos según una variable categórica. Útil para comparación por grupos.

alkemy

PairGrid: Ejemplo

PairGrid ofrece mayor flexibilidad que pairplot, permitiendo aplicar diferentes funciones a diferentes partes de la matriz:

- Diagonal: histogramas, KDE, etc.
- Triángulo superior: un tipo de gráfico
- Triángulo inferior: otro tipo de gráfico

```
g = sns.PairGrid(df, hue='genero')
g.map_diag(sns.histplot)
g.map_offdiag(sns.scatterplot)
g.add_legend()
```

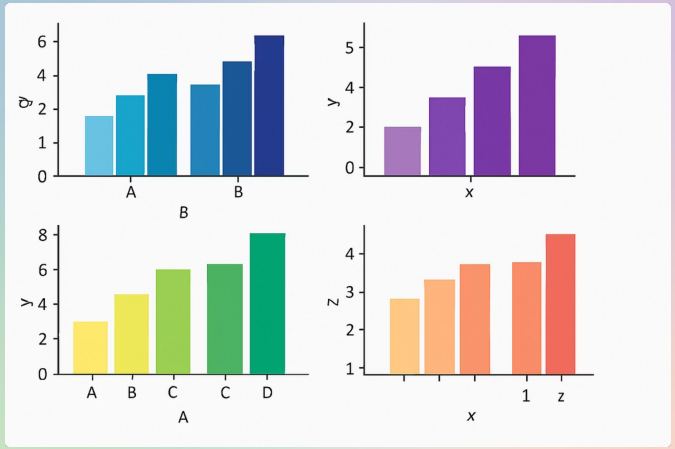
FacetGrid: Ejemplo

```
g = sns.FacetGrid(df, col='genero')
g.map(sns.histplot, 'edad')
```

También se puede usar con row, col, hue para crear estructuras más complejas.

Función	Diferencias clave
FacetGrid	Divide por categorías (columnas/filas)
PairGrid	Visualización cruzada de variables numéricas

alkemy



Personalización avanzada de gráficos

Paletas de colores

Seaborn ofrece paletas predefinidas como 'deep', 'muted', 'pastel', 'bright', 'dark', y 'colorblind'.

Estilos

Cambia el aspecto general con sns.set_style() usando opciones como 'whitegrid', 'darkgrid', 'white', 'dark', y 'ticks'.

Contextos

Ajusta el tamaño con sns.set_context() usando 'paper', 'notebook', 'talk', o 'poster'.

Matplotlib

Accede a la figura y ejes subyacentes para personalización adicional.

alkemy

Casos de uso comunes

Análisis exploratorio

Utiliza pairplot y histplot para entender la distribución y relaciones entre variables.

Análisis estadístico

Emplea boxplot y violinplot para comparar distribuciones entre grupos.

Correlaciones

Usa heatmap para visualizar matrices de correlación y detectar relaciones.

Presentación

Refina los gráficos con paletas personalizadas y estilos para informes y presentaciones.

alkemy

Live Coding

¿En qué consistirá la Demo?

Se realizará un recorrido visual sobre diferentes gráficos de Seaborn para modelar y analizar visualmente la relación entre dos o más variables.

1. *Importe dataset tips desde Seaborn.*
2. *Cree histogramas con histplot() para explorar variables numéricas.*
3. *Use regplot() para modelar regresión lineal simple.*
4. *Explore relaciones múltiples con pairplot() y categoría con hue.*
5. *Compare boxplot y violinplot para variables categóricas.*
6. *Visualice matriz de correlación con heatmap().*
7. *Explique cómo interpretar los resultados visuales en contexto real.*

Tiempo: 25 Minutos

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio

Visualizando la Propina: Un Análisis con Seaborn

Visualizando la Propina: Un Análisis con Seaborn

Contexto: 🙌

Un equipo de análisis de datos desea evaluar cómo varía el monto de propina en función de distintos factores como el total de la cuenta, el día de la semana, el tamaño del grupo y el género del cliente. Se espera un informe gráfico que explore estas relaciones mediante visualización con Python.

Consigna: ✍️

Utilizando el dataset tips (disponible en Seaborn), genere al menos 4 visualizaciones que representen relaciones significativas entre variables. Incluye un gráfico de regresión que explore la dependencia entre dos variables numéricas. Personalice los gráficos con paletas, estilos y anotaciones claras.

Tiempo 🕐**:** *45 Minutos*

Visualizando la Propina: Un Análisis con Seaborn

Paso a paso: ⚙️

1. Cargue el dataset tips de Seaborn.
2. Cree un histograma para total_bill o tip.
3. Genere un regplot para total_bill vs tip.
4. Cree un boxplot de tip según sex o day.
5. Use pairplot() para analizar múltiples variables.
6. Muestre un heatmap() con la matriz de correlación.
7. Personalice colores y estilos con sns.set_style() y palette.,

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Exploramos la biblioteca Seaborn para análisis visual de datos.***
- ✓ ***Generamos múltiples tipos de gráficos (histograma, scatterplot, regplot, heatmap).***
- ✓ ***Interpretamos relaciones entre variables a través de visualización.***
- ✓ ***Implementamos gráficos con personalización avanzada (hue, style, col, palettes).***
- ✓ ***Analizamos visualmente la dependencia entre variables numéricas y categóricas.***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy